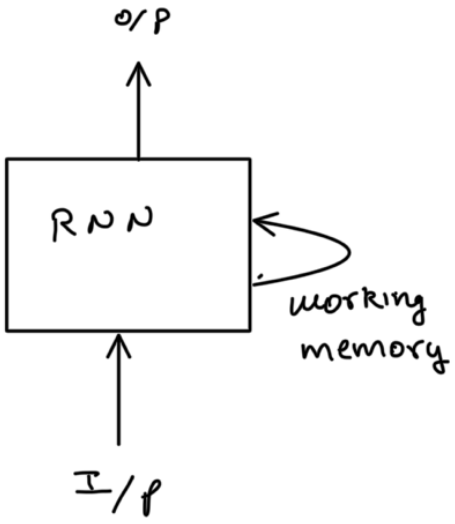
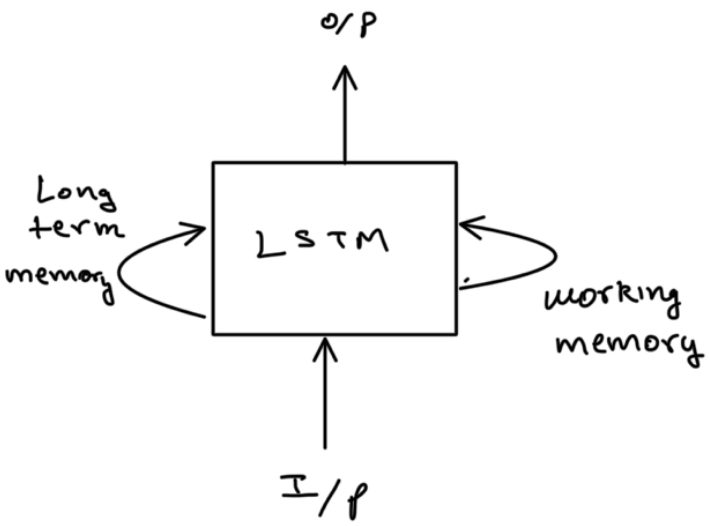
LSTM Derivation





This above shown is the major architectural difference between RNN & LSTM

The limitation for vanilla RNN is that it fails to preserve info for a longer interval of time step's.

So, This is the central idea of LSTM for a mechanism to help retain information for a longer period of time.

The main Reason RNN struggle with long term dependencies is due to :

① Vanishing gradient

② Exploding gradient

As, we know from our previous derivation

recurrent weight = 0.5

$$\frac{\partial h_t}{} = 0.5(1 - h_i^2)$$

$$\frac{\partial h_t}{\partial h_{t-1}}$$

for easy numerical illustration, take derivative at each step 0.5

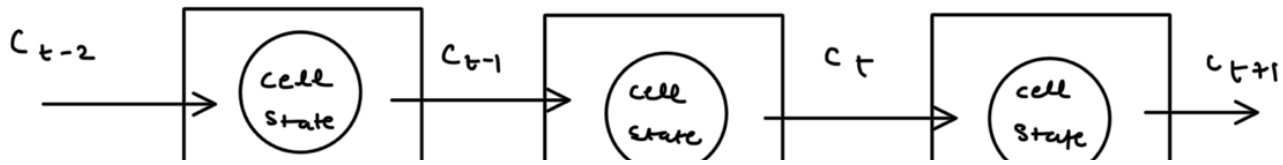
$t=1$	0.5
$t=2$	$0.5^2 = 0.25$
$t=3$	$0.5^3 = 0.125$
$t=4$	$0.5^4 = 0.0625$
⋮	
$t=10$	$0.5^{10} = 0.000977..$

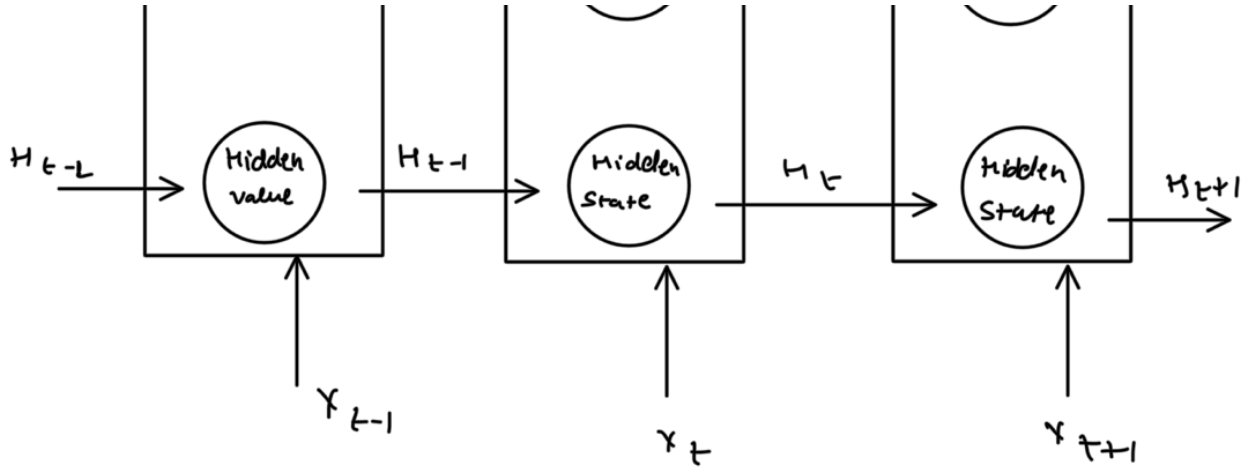
(*) And after decreasing gradient & multiplying it by a learning rate (which is <1) makes the updates in the parameter's negligible.

So, due to this as the sentence increases the dependencies on previous words decreases due to vanishing gradient.

but if we multiply the gradient with a number to prevent it from vanishing it could cause exploding grad problem.

LSTM Architecture





Constant error carousel (CEC) {original paper concept}

if let's say $c_{t-1} = 5$

at next step we want to preserve this info.

$$c_t = c_{t-1}$$

then,

$$c_t = 5$$

at next time step

$$c_{t+1} = 5$$

the memory is carried forward without any transformation

so,

$$\frac{\partial c_t}{\partial c_{t-1}} = 1$$

$$\frac{\partial c_{10}}{\partial c_0} = 1 \times 1 \times 1 \times 1 \times \cdots \times 1 = 1$$

Imagine, if the cell receives some info at every step

even if it is unwanted it would accumulate at every step.

$$c_0 = 5$$

$$c_1 = 5 + 2 = 7$$

$$c_2 = 7 + 3 = 10$$

$$c_3 = 10 + 4 = 14$$

So, we introduce a gate in our architecture to control it; that what should be kept or ignored.

Let the gate o/p be:

$$i_t \in (0, 1)^m$$

it must depend on current I/p & previous context:

$$i_t = f(x_t, h_{t-1})$$

for current I/p:

$$z_x = W_i x_t$$

for previous hidden State:

$$z_h = U_i h_{t-1}$$

Combining both we get;

$$\therefore \quad z_i = W_i x_t + U_i h_{t-1} + b_i$$

As the gate should lie b/w 0 & 1 we would apply activation function on it. (Sigmoid)

$$\sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

$$\therefore \boxed{i_t = \sigma(w_i x_t + u_i h_{t-1} + b_i)}$$

Now we calculate the actual information that could enter the cell state.

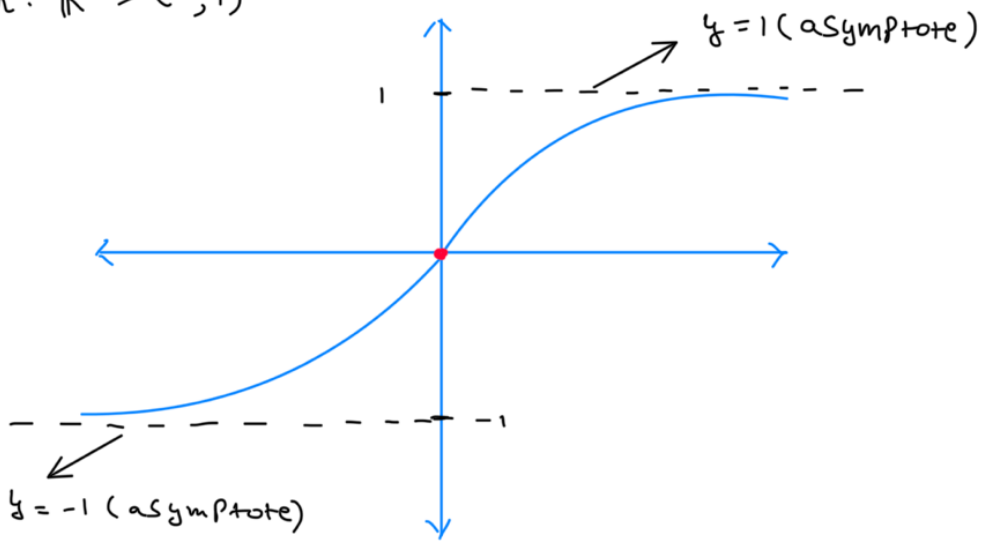
we call it candidate memory, represented by:

$$\tilde{c}_t = f(x_t, h_{t-1})$$

$$z_t = w_c x_t + u_c h_{t-1} + b_c$$

we would apply tanh activation on z_t, to keep it signed i.e +ve for +ve i/p & -ve for -ve I/p & smooth curve easily differentiated.

$\tanh : \mathbb{R} \rightarrow (-1, 1)$



$$-1 < \tanh(x) < 1 \quad , \qquad \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$\therefore \quad \boxed{\tilde{c}_t = \tanh(z_t)}$$

So, now the problem is that not every new candidate is usefull.

now, we would perform element wise product (Hadamard product) of $i_t \delta \tilde{c}_t$

So, the accepted candidate update as:

$$\therefore \quad \boxed{u_t = i_t \odot \tilde{c}_t}$$

$$u_t = \begin{bmatrix} i_{t,1}\ \tilde{c}_{t,1} \\ i_{t,2}\ \tilde{c}_{t,2} \\ \vdots \\ i_{t,m}\ \tilde{c}_{t,m} \end{bmatrix}_{m \times 1}$$

We got an i/p to our long term state, an i/p that is filtered.

but we have a problem to address, i.e

we can't retain everything forever. Some previous info becomes irrelevent as new info arrives,

So, we introduce forget gate for it.

we should calculate forget gate based on previous memory state & current I/P

we first combine both we get:

$$z_f = W_f x_t + U_f h_{t-1} + b_f$$

△ then apply sigmoid to it. (same reason as before it)

$$\therefore \quad \boxed{f_t = \sigma(z_f)}$$

Applying f_t to previous state:

$$\boxed{r_t = f_t \odot c_{t-1}}$$

$$r_t = \begin{bmatrix} f_{t,1} \cdot c_{t-1,1} \\ \vdots \\ f_{t,m} \cdot c_{t-1,m} \end{bmatrix}_{m \times 1}$$

now we have two seperate contributon's:

$$r_t = f_t \odot c_{t-1} \quad \{\text{Retained old memory}\}$$

$$u_t = i_t \odot \tilde{c}_t \quad \{\text{Accepted new information}\}$$

The central memory update equation of Mordern LSTM

★

$$c_{t,j} = f_{t,j} \odot c_{t-1,j} + i_{t,j} \odot \tilde{c}_{t,j}$$

where, $j = 1, \ldots m$

now we need a vector which controls how much of stored info becomes visible to the next time step as a hidden state

Let it be:

$$o_t \in \mathbb{R}^{m \times 1}$$

$$o_t = g(x_t, h_{t-1})$$

$$z_o = W_o x_t + U_o h_{t-1} + b_o$$

applying sigmoid (σ) to z_o {Same reason as explained before}

$$\boxed{o_t = \sigma(z_o)}$$

now, we restrict cell state value to $(-1, 1)$ as explained before by apply tanh to it.

$$\tanh(c_t)$$

& then multiplying it by output gate.

$$\boxed{h_t = o_t \odot \tanh(c_t)}$$ hidden state of current time step.

$$h_t \in \mathbb{R}^{m \times 1}$$

The output layer converts h_t into o/p probability.

$$z_t = W_t h_t + b_y$$

we have to apply Softmax to z_t to convert logit into probability

for K classes.

$$p_{t,k} = \frac{e^{z_{t,k}}}{\sum_{j=1}^{K} e^{z_{t,j}}}$$

this gives the probability vector of each class i.e K for the given time step.

Actual target is

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} \in \mathbb{R}^{K \times 1}$$

$$y_k \in \{0, 1\}, \quad \sum_{k=1}^{K} y_k = 1$$

Predicted probability vector is:

$$\hat{p} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_K \end{bmatrix} \in \mathbb{R}^{K \times 1}$$

$$0 < p_k < 1 \quad , \qquad \sum_{k=1}^{K} p_k = 1$$

$$P(y|p) = \prod_{k=1}^{K} p_k^{y_k}$$

we convert probability into a loss function:

$$L = \log\left(P(y|p)\right)$$

$$\boxed{L = -\sum_{k=1}^{K} y_k \log(p_k)}$$

This is called cross entropy loss.

Now we perform Backpropagation through time (BPTT)

we would calculate loss because of logit's.

$$\partial L = \sum^{K} \frac{\partial L}{\partial p_i}$$

$$\frac{\partial L}{\partial z_k} = \sum_{j=1} \frac{\partial p_j}{\partial z_k}$$

$$\frac{\partial L}{\partial p_j} = -\frac{y_j}{p_j} \quad \& \quad \frac{\partial p_j}{\partial z_k} = p_j (\delta_{jk} - p_k)$$

Substituting both we get:

$$\frac{\partial L}{\partial z_k} = -\sum_{j=1}^{k} y_j (\delta_{jk} - p_k)$$

$$= -y_k + p_k \sum_{j=1}^{k} y_j \rightarrow 1$$

$$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$$

we know from forward prop.

$$z = W_y h_t + b_y$$

Change in loss due to b_y

$$\frac{\partial L}{\partial b_{y,k}} = \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial b_{y,k}}$$

$$\frac{\partial z_k}{\partial b_{y,k}} = 1$$

Substituting in the above eqn

$$\frac{\partial L}{\partial b_{j,k}} = \frac{\partial L}{\partial z_k} \times 1 = p - y$$

$$\boxed{\frac{\partial L}{\partial b_{j,k}} = p - y}$$

Change in Loss due to w_j

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial w_j}$$

$$\frac{\partial z_k}{\partial w_{j,kj}} = h_{t,j}$$

we get

$$\boxed{\frac{\partial L}{\partial w_j} = (p - y)\, h_t^T}$$

Similarly we get for h_t i.e

$$\boxed{\frac{\partial L}{\partial h_t} = w_j^T (p - y)}$$

Recall it's forward eqn:

$$h_t = o_t \odot \tanh(C_T)$$

$$\frac{\partial L}{\partial o_t} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial o_t}$$

for,

$$\frac{\partial h_t}{\partial o_t} = \tanh(c_t)$$

$$\boxed{\frac{\partial L}{\partial o_t} = W_y^T (P - y) \odot \tanh(c_t)}$$

As, we know

$$\frac{d}{dx}(\tanh x) = 1 - \tanh^2(x)$$

So, similarly

$$\boxed{\frac{\partial L}{\partial c_t} = W_y^T (P - y) \odot o_t \odot (1 - \tanh^2(c_t))}$$

gradient through forget gate:

we calculated the eqn

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$\frac{\partial c_t}{\partial f_t} = c_{t-1}$$

$$\frac{\partial L}{\partial f_t} = \frac{\partial L}{\partial c_t} \odot c_{t-1}$$

▷ Similarly with help of eqn $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$

we also get,

$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \odot f_t$$

$$\frac{\partial L}{\partial i_t} = \frac{\partial L}{\partial c_t} \odot \tilde{c}_t$$

$$\frac{\partial L}{\partial \tilde{c}_t} = \frac{\partial L}{\partial c_t} \odot i_t$$

▷ $o_T = \sigma(z_o)$

$$z_o = W_o x_t + U_o h_{t-1} + b_o$$

$$\sigma'(z_o) = \sigma(z_o)(1 - \sigma(z_o))$$

we obtain

$$\frac{\partial L}{\partial z_o} = \frac{\partial L}{\partial o_t} \odot o_t \odot (1 - o_t)$$

and from eq^n $z_o = W_o x_t + U_o h_{t-1} + b_o$ we could easily derive that

$$\frac{\partial L}{\partial W_o} = \frac{\partial L}{\partial z_o} x_t^T$$

$$\frac{\partial L}{\partial U_o} = \frac{\partial L}{\partial z_o} h_{t-1}^T$$

$$\frac{\partial L}{\partial b_o} = \frac{\partial L}{\partial z_o}$$

we known from before

$$z_t = W_c x_t + U_c h_{t-1} + b_c$$

$$\tilde{c}_t = \tanh(z_t)$$

so,

$$\frac{\partial L}{\partial z_c} = \frac{\partial L}{\partial \tilde{c}_t} \odot (1 - \tilde{c}_t^2)$$

$$\frac{\partial L}{\partial z_t} \qquad \frac{\partial L}{\partial \tilde{c}_t}$$

So, with similar Calculation we can say

$$\frac{\partial L}{\partial w_c} = \frac{\partial L}{\partial z_t} x_t^T$$

$$\frac{\partial L}{\partial w_c} = \frac{\partial L}{\partial z_t} h_{t-1}^T$$

$$\frac{\partial L}{\partial b_c} = \frac{\partial L}{\partial z_t}$$

As, we move back

we know that

$$f_t = \sigma(z_f)$$

$$\frac{\partial L}{\partial z_f} = \frac{\partial L}{\partial f_t} \odot f_t \odot (1 - f_t)$$

- Similarly

$$\frac{\partial L}{\partial w_f} = \frac{\partial L}{\partial f_t} x_t^T$$

$$\frac{\partial L}{\partial U_f} = \frac{\partial L}{\partial f_t} h_{t-1}^T$$

$$\frac{\partial L}{\partial b_f} = \frac{\partial L}{\partial f_t}$$

for i/p gate

$$i_t = \sigma(z_i)$$

where, $z_i = W_i x_t + U_i h_{t-1} + b_i$

$$\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial i_t} \odot i_t \odot (1 - i_t)$$

Similar to i_t

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial z_i} x_t^T$$

$$\frac{\partial L}{\partial U_i} = \frac{\partial L}{\partial z_i} h_{t-1}^T$$

$$\frac{\partial L}{\partial b_i} = \frac{\partial L}{\partial z_i}$$

Gradient arriving at previous hidden step

So,

$$\frac{\partial L}{\partial h_{t-1}} = U_i^T \frac{\partial L}{\partial z_i} + U_f^T \frac{\partial L}{\partial z_f} + U_o^T \frac{\partial L}{\partial z_o} + U_c^T \frac{\partial L}{\partial z_c}$$

As, all these eq^{n}/s contribute to it.

- we have calculated

$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \odot f_t$$

but full c_{t-1} grad is still to be calculated.

we know $h_{t-1} = o_{t-1} \odot \tanh(c_{t-1})$

grad arriving at c_{t-1} also contribute

So, $$\frac{\partial L}{c_{t-1}} = \frac{\partial L}{\partial h_{t-1}} \odot o_{t-1} \odot (1 - \tanh^2(c_{t-1}))$$

finally we get,

$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \odot f_t + \frac{\partial L}{\partial h_{t-1}} \odot o_{t-1} \odot (1 - \tanh^2(c_{t-1}))$$

Now we do the parameter update

all update parameter's are.

$$W_j, \quad U_j, \quad b_j$$

where $j \in \{i, f, o, c\}$

A update eqn is

$$\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$$

now we would again perform the same step's with updated parameters. i.e forward prop, Backward prop, weight update.